



LEAPPs CONTRIBUTOR GUIDE

How to Write an iLEAPP Module

The LEAPPs ecosystem runs on contributions. A module is a single Python file, and this guide walks through everything: module structure, SQLite and plist parsing, timestamps, media, LAVA conversation view, the checks pull requests run, and submitting one.

Alexis Brignoni
Updated September 14, 2026

Live version: leapps.org/blog-post?post=2026-06-14-how-to-write-an-ileapp-module

The LEAPPs ecosystem runs on contributions. Every artifact module in iLEAPP, ALEAPP, RLEAPP, VLEAPP, and DLEAPP was written by someone who looked at a data source, figured out what it contained, and wrote the code to surface it. That is how the tools grew from a handful of modules to the hundreds that exist today. It is also how they will keep growing.

This post is for anyone who wants to add to that. Whether you are a developer who has never touched a forensics tool or an examiner who writes Python but has never contributed to an open source project, the barrier is lower than you might think. A module is a single Python file. The framework handles all the output: HTML reports, TSV exports, timeline entries, and the LAVA database. Your job is to find the data and return it in a consistent format.

The guide below walks through everything: the module structure, the metadata block that drives the framework, parsing SQLite databases and plists, handling timestamps and media, wiring LAVA conversation view for messaging artifacts, the checks that run on pull requests, and submitting one. It includes a complete working example you can use as a starting point.

One note before you dive in. This is a working guide, not a finished specification. The LEAPPs tools and LAVA are under active development, and the contributor workflow will continue to evolve. A dedicated documentation site is in progress that will house always-current documentation for iLEAPP, ALEAPP, RLEAPP, VLEAPP, and LAVA in one place. Until that site is live, this post is the best starting point. For anything that lands after the update date below, check the iLEAPP repository's own documentation in `admin/docs`.

Updated September 14, 2026. This revision covers Tabler icon names, media through `check_in_media`, the real paths of the files you read as the third return value, the column order for conversation artifacts, and the checks pull requests run. The example modules in this post were run against iLEAPP as of that date.

A downloadable PDF version of this guide is available here: [iLEAPP Module Contributor Guide \(PDF\)](#)

With that said, pick an artifact, open a file, and write the module. The community benefits every time someone does.

Note: This guide uses iLEAPP as its reference. ALEAPP, RLEAPP, VLEAPP, and DLEAPP use the same `__artifacts_v2__` format and the same `@artifact_processor` decorator, but their helper functions are not identical. ALEAPP, for example, has no `get_plist_file_content` or `convert_cocoa_core_data_ts_to_utc`. Before you import a helper in another tool, check that it exists in that tool's `scripts/ilapfuncs.py`, and read that repository's workflows for the checks its pull requests run. The artifact paths are specific to each platform.

What a Module Is

Every artifact iLEAPP parses comes from a module: a single Python file in `scripts/artifacts/`. iLEAPP loads the modules in that folder automatically at runtime. There is no registry to update and no configuration file to edit. Drop a valid module file in the folder and iLEAPP picks it up.

Each module defines what files to look for, how to parse them, and what data to return. The framework handles all output: HTML report, TSV, timeline, KML when you ask for it, and the LAVA database.

Prerequisites

- Python 3.10 to 3.14
- A local clone of the iLEAPP repository: <https://github.com/abrignoni/iLEAPP>
- Dependencies installed: `pip install -r requirements.txt`
- A test extraction or sample image to run against

Module Structure

A module has two required parts: the `__artifacts_v2__` dictionary and the processing function.

1. `__artifacts_v2__`

Put this dictionary at the top of the file, before the imports. The loader reads it by name, so the position is a convention rather than a rule, but it is where nearly all modules keep it and where the next person will look for it. It tells iLEAPP everything it needs to know about the artifact before parsing begins.

```
__artifacts_v2__ = {
    "myArtifact": {
        "name": "My Artifact",
        "description": "Messages stored by MyApp",
        "author": "@YourHandle",
        "creation_date": "2026-09-14",
        "last_update_date": "2026-09-14",
        "requirements": "none",
        "category": "Social Media",
        "notes": "",
        "paths": ('*/mobile/Containers/Data/Application/*/Library/myapp.db',),
        "output_types": "standard",
        "artifact_icon": "message-circle"
    }
}
```

Field reference:

Field	Description
"myArtifact" (key)	Must exactly match the processing function name
name	Display name in reports and LAVA. Loading fails if two artifacts in the folder share a name
description	One line saying what the artifact reports. CI fails a description that is missing, runs to more than one line, only repeats the name, or duplicates another artifact's in the same module
author	Your name or handle, for example @YourHandle
creation_date	Date you wrote it (YYYY-MM-DD)
last_update_date	Date of last change (YYYY-MM-DD)
requirements	A note on anything the artifact needs, or "none". iLEAPP does not read it
category	Category grouping in reports and LAVA
notes	Research references, caveats, or an empty string. It is written into LAVA, so say only what the data shows
paths	Tuple of file path patterns (see below)
output_types	Which outputs to write (see below)
artifact_icon	Tabler icon name for the report sidebar and LAVA
sample_data	Optional. Where the artifact was tested and what each test produced (see below)

`sample_data` is a dictionary of short notes, one per test source. CI checks that every key and value is a non-empty string.

```
"sample_data": {
    "my_test_image": "MyApp 4.2 | 12 rows",
```

```
}
```

output_types **values:**

Value	Output generated
"standard"	HTML, TSV, timeline, and LAVA. Use this for most artifacts
"all"	All of the above plus KML
"lava_only"	LAVA database only, with no HTML report, TSV, or timeline
"none"	No output (used for modules that only collect device info)
List e.g. ["html", "lava"]	Specific combination

If you leave output_types out, every output is written, KML included. A KML file is only written when the headers include columns named exactly Latitude and Longitude.

paths **patterns:**

Patterns are matched with Python's fnmatch, not glob, so a * can match across / and is not limited to one folder level. Matching is case-sensitive on macOS and Linux and case-insensitive on Windows, for every input type.

```
# Single file
"paths": ('*/mobile/Library/SMS/sms.db',)

# Multiple patterns
"paths": (
    '*/mobile/Library/CallHistoryDB/CallHistory.storedata',
    '*/mobile/Library/CallHistoryDB/call_history.db',
)

# Match all files in a directory
"paths": ('*/mobile/Library/SMS/Attachments/*',)

# A database together with its -wal and -shm files
"paths": ('*/mobile/Library/SMS/sms.db*',)

# Two spellings of one name: use a bracket class, never two patterns
"paths": ('*/Library/Preferences/com.apple.[Mm]obileSMS.plist',)
```

CI rejects two patterns in one tuple that differ only in case, and the same pattern listed twice.

artifact_icon **values:**

Use an icon name from Tabler Icons (<https://tabler.io/icons>) that also appears in iLEAPP's bundled copy, scripts/_elements/tabler-icons.css, which is version 3.44.0 as of this update. Common choices: "message-circle", "phone-call", "user", "map-pin", "clock", "wifi", "camera", "file". Older modules use Feather icon names, which the HTML report still maps to their Tabler equivalents, but new modules should use Tabler names.

2. The Processing Function

The function name must match the key in __artifacts_v2__. It takes a single context argument and returns a tuple of (data_headers, data_list, source_path).

```
from scripts.ilapfuncs import (
    artifact_processor,
    convert_unix_ts_to_utc,
    get_file_path,
    get_sqlite_db_records,
)
```

```

@artifact_processor
def myArtifact(context):
    files_found = context.get_files_found()
    source_path = get_file_path(files_found, "myapp.db")
    data_list = []
    if not source_path:
        return (), data_list, ''

    query = '''
    SELECT timestamp, sender, message
    FROM messages
    '''

    for record in get_sqlite_db_records(source_path, query):
        data_list.append((convert_unix_ts_to_utc(record[0]), record[1], record[2]))

    data_headers = (
        ('Timestamp', 'datetime'),
        ('Sender',
         'Message'
        )
    )
    return data_headers, data_list, source_path

```

The `@artifact_processor` decorator handles all output generation. Your function only needs to return clean data.

Returning Data

`data_headers`

A tuple defining column names. Plain strings for text columns. Tuples of (name, type) for columns that need special handling.

```

data_headers = (
    ('Timestamp', 'datetime'),      # UTC datetime from a conversion helper
    ('Birthday', 'date'),          # Calendar date with no time of day
    ('Sender',                      # Plain text
     'Message'
    ),
    ('Attachment', 'media'),        # Reference returned by check_in_media
    ('Phone Number', 'phonenumber'), # Phone number string
)

```

Special column types:

Type	Use for
datetime	A timezone-aware UTC datetime from one of the conversion helpers. LAVA stores it as a Unix timestamp
date	A calendar date with no time of day. LAVA stores it as YYYY-MM-DD
media	The reference returned by <code>check_in_media</code> or <code>check_in_embedded_media</code> , not a file path (see Media below)
phonenumber	Phone number strings. LAVA has a renderer for this type

These four are the types the framework and LAVA act on, so do not invent new ones. Another type name is ignored when the column is stored and displayed, but it is still written into the LAVA manifest, where it would take effect the day LAVA starts handling that name.

A local time with no recorded time zone cannot be converted to UTC. Keep it as a text column rather than typing it datetime, because a naive value in a datetime column is treated as UTC.

data_list

A list of tuples. Each tuple is one row. Values must be in the same order as data_headers.

```
from datetime import date, datetime, timezone
```

```
data_list = [
    (datetime(2023, 4, 15, 17, 30, 9, tzinfo=timezone.utc), date(1990, 5, 1),
     "Alice", "See you tomorrow", media_ref, "+19195551234"),
    (datetime(2023, 4, 15, 17, 35, 0, tzinfo=timezone.utc), None,
     "Bob", "Confirmed", None, "+19195559876"),
]
```

Null or missing values should be None.

source_path

The third value is the path of the file the data came from. The decorator splits it on newlines and turns each line into a path relative to the extraction, which is what the report's "located at" line shows. If you read more than one file, return every path you read, one per line. Return an empty string only from a branch where no file was found. A file that exists but lacks the table you expected still returns its path.

```
# One file
return data_headers, data_list, source_path

# Several files: every path you read, one per line
return data_headers, data_list, '\n'.join(files_read)
```

If a row needs its own source file column, pass the path through `context.get_relative_path()` before adding it to the row. Only the third return value is shortened for you, so a full path placed in a row would put the examiner's own folder names in the report.

```
data_list.append((timestamp, name, context.get_relative_path(file_found)))
```

CI checks both: `check_source_path.py` rejects prose in place of a path, and `check_report_local_paths.py` looks for full paths in rows.

Parsing SQLite Databases

Many iOS artifacts are in SQLite databases. Use `get_sqlite_db_records` from `ilapfuncs`. It opens the database read-only and returns rows you can read by position or by column name. When a column only exists in some versions of an app, check for it with `does_column_exist_in_db` instead of letting the query fail.

```
from scripts.ilapfuncs import (
    artifact_processor,
    convert_cocoa_core_data_ts_to_utc,
    does_column_exist_in_db,
    get_file_path,
    get_sqlite_db_records,
)

@artifact_processor
def myArtifact(context):
    files_found = context.get_files_found()
    source_path = get_file_path(files_found, "myapp.db")
    data_list = []
```

```

if not source_path:
    return (), data_list, ''

# ZREAD only exists in later versions of this app
read_column = 'ZREAD' if does_column_exist_in_db(source_path, 'ZMESSAGE', 'ZREAD') else 'NULL'

query = f'''
SELECT
    ZMESSAGE_DATE,
    ZSENDER,
    ZBODY,
    {read_column} AS ZREAD
FROM ZMESSAGE
ORDER BY ZMESSAGE_DATE
'''

for record in get_sqlite_db_records(source_path, query):
    data_list.append((
        convert_cocoa_core_data_ts_to_utc(record['ZMESSAGE_DATE']),
        record['ZSENDER'],
        record['ZBODY'],
        record['ZREAD'],
    ))

data_headers = (
    ('Timestamp', 'datetime'),
    ('Sender',),
    ('Message',),
    ('Read',)
)
return data_headers, data_list, source_path

```

Parsing Plist Files

For plist-based artifacts, use `get_plist_file_content`. It decodes `NSKeyedArchiver` plists for you. When the file cannot be read, it logs the error and returns an empty dictionary.

```

from scripts.ilapfuncs import (
    artifact_processor,
    convert_plist_date_to_utc,
    get_file_path,
    get_plist_file_content,
)

@artifact_processor
def myArtifact(context):
    files_found = context.get_files_found()
    source_path = get_file_path(files_found, "com.example.myapp.plist")
    data_list = []
    if not source_path:
        return (), data_list, ''

    pl = get_plist_file_content(source_path)
    if isinstance(pl, dict):
        for entry in pl.get('entries', []):
            data_list.append((convert_plist_date_to_utc(entry.get('date')), entry.get('value', '')))

    data_headers = (
        ('Date', 'datetime'),
        ('Value',)
    )
    return data_headers, data_list, source_path

```

Timestamp Conversion

Store timestamps as UTC. The helpers below return timezone-aware datetime objects, ready for a datetime column.

```
from scripts.ilapfuncs import (
    convert_cocoa_core_data_ts_to_utc, # Cocoa/Core Data timestamp (seconds since 2001-01-01)
    convert_unix_ts_to_utc,            # Unix timestamp in seconds, milliseconds,
                                      # microseconds or nanoseconds
    convert_plist_date_to_utc,         # Plist date, which is already UTC
    convert_ts_human_to_utc,           # 'YYYY-MM-DD HH:MM:SS' string that is already UTC
)
```

convert_unix_ts_to_utc works out the unit from the size of the number. Close to 1970 that guess cannot tell the units apart, so when you know the unit, convert the value to seconds yourself first.

Media

Attachments, photos, and other files belong in the report as media rather than as a file name. Pass the file to check_in_media and put the reference it returns in a column typed media. The HTML report and LAVA then show the media instead of a bare file name.

```
from scripts.ilapfuncs import check_in_media, get_file_path

attachment_path = get_file_path(files_found, "Attachments/photo_0001.jpg")
media_ref = check_in_media(attachment_path, "photo_0001.jpg") if attachment_path else None
```

The file has to be one of the files your paths patterns matched, so add a pattern for the folder the attachments live in. When the media is stored inside a database as bytes, use check_in_embedded_media(source_file, data, name) instead. Both return None when there is nothing to check in.

Adding LAVA Conversation View

If your artifact contains messages, add data_views to the __artifacts_v2__ definition to enable conversation view in LAVA.

```
__artifacts_v2__ = {
    "myMessaging": {
        "name": "My Messaging App",
        "description": "Messages stored by MyApp",
        "author": "@YourHandle",
        "creation_date": "2026-09-14",
        "last_update_date": "2026-09-14",
        "requirements": "none",
        "category": "Messaging",
        "notes": "",
        "paths": ('*/mobile/Containers/Data/Application/*/Library/myapp.db', ),
        "output_types": "standard",
        "artifact_icon": "message-circle",
        "data_views": {
            "conversation": {
                "conversationDiscriminatorColumn": "Thread ID",
                "conversationLabelColumn": "Contact Name",
                "textColumn": "Message",
                "directionColumn": "Direction",
                "directionSentValue": "Sent",
                "timeColumn": "Timestamp",
                "senderColumn": "Sender",
            }
        }
    }
}
```

```

        "mediaColumn": "Attachment"
    }
}
}
}

```

The column names in `data_views` must match the display names defined in `data_headers`, not the raw SQL column names.

Conversation view fields:

Field	Description
<code>conversationDiscriminatorColumn</code>	Column that groups messages into threads
<code>conversationLabelColumn</code>	Optional. Column used to label threads in the sidebar
<code>textColumn</code>	Column containing message text
<code>directionColumn</code>	Column indicating sent vs received
<code>directionSentValue</code>	Value in <code>directionColumn</code> that means sent (string or integer)
<code>timeColumn</code>	Column containing message timestamp
<code>senderColumn</code>	Optional. Column containing sender identity for received messages
<code>sendMessageLabelColumn</code>	Optional. Column to use as sender label for sent messages
<code>sendMessageStaticLabel</code>	Optional. Fixed string label for sent messages (e.g. "This Phone")
<code>mediaColumn</code>	Optional. Column containing the media reference

Before you declare `directionSentValue`, establish from a source what the stored value means. It decides which side of the conversation view a message appears on.

Column order. A conversation artifact puts the columns an examiner reads first at the front, in this order: the `timeColumn`, any other `datetime` or `date` columns, then the `direction`, `sender`, `conversation label`, `message text`, and `media` columns, then everything else. A role you do not declare is skipped. `check_conversation_column_order.py` enforces this in CI. The values in each row line up with the headers by position, so when you move a header, move its value in the row (or its column in the `SELECT`) in the same edit.

Complete Module Example

A full working module for a hypothetical SQLite-based messaging app with attachments.

```

__artifacts_v2__ = {
    "exampleMessages": {
        "name": "Example App - Messages",
        "description": "Messages stored by Example App, with their attachments",
        "author": "@YourHandle",
        "creation_date": "2026-09-14",
        "last_update_date": "2026-09-14",
        "requirements": "none",
        "category": "Messaging",
        "notes": "",
        "paths": (
            '*mobile/Containers/Data/Application/*/Library/Application Support/example.db*',
            '*mobile/Containers/Data/Application/*/Library/Application Support/Attachments/*',
        ),
        "output_types": "standard",
        "artifact_icon": "message-circle",
        "data_views": {

```

```

        "conversation": {
            "conversationDiscriminatorColumn": "Thread ID",
            "conversationLabelColumn": "Contact",
            "textColumn": "Message",
            "directionColumn": "Direction",
            "directionSentValue": "Sent",
            "timeColumn": "Timestamp",
            "senderColumn": "Contact",
            "mediaColumn": "Attachment"
        }
    }
}

from scripts.ilapfuncs import (
    artifact_processor,
    check_in_media,
    convert_unix_ts_to_utc,
    get_file_path,
    get_sqlite_db_records,
)

@artifact_processor
def exampleMessages(context):
    files_found = context.get_files_found()
    source_path = get_file_path(files_found, "example.db")
    data_list = []
    if not source_path:
        return (), data_list, ''

    query = '''
    SELECT
        m.timestamp,
        CASE m.direction WHEN 1 THEN 'Sent' ELSE 'Received' END,
        c.display_name,
        m.body,
        m.attachment_name,
        m.thread_id
    FROM messages m
    LEFT JOIN contacts c ON m.contact_id = c.id
    ORDER BY m.timestamp
    '''

    for record in get_sqlite_db_records(source_path, query):
        attachment = None
        if record[4]:
            attachment_path = get_file_path(files_found, f"Attachments/{record[4]}")
            if attachment_path:
                attachment = check_in_media(attachment_path, record[4])
        data_list.append((
            convert_unix_ts_to_utc(record[0]),
            record[1], # direction
            record[2], # contact
            record[3], # message body
            attachment,
            record[5], # thread id
        ))

    data_headers = (
        ('Timestamp', 'datetime'),
        ('Direction',
        'Contact',
        'Message',
        ('Attachment', 'media')),
    )

```

```

        'Thread ID',
    )
    return data_headers, data_list, source_path

```

Testing Your Module

Run iLEAPP against a test extraction and verify your artifact appears in the output.

CLI:

```
python ileapp.py -t fs -i /path/to/extraction -o /path/to/output
```

-t also accepts tar, zip, gz, itunes, file, and raw, where raw reads a disk image or an E01 acquisition in place.

To run only your artifact, list it in a profile file and pass the profile with -m:

```
{"leapp": "ileapp", "format_version": 1, "plugins": ["myArtifact"]}
```

```
python ileapp.py -t fs -i /path/to/extraction -o /path/to/output -m myArtifact.ilprofile
```

GUI:

```
python ileappGUI.py
```

Check the output folder for:

- An HTML report page for your artifact
- A TSV file with the same data
- `_lava_artifacts.db`, the LAVA database holding your artifact's table
- `_lava_data.lava`, the file LAVA opens, with your artifact's definition

Verify the LAVA output by opening the project in LAVA and confirming your artifact appears in the correct category with the correct columns.

Read the run log as well. A module that hits a database error logs it (for example `no such column`) and the run still finishes, so a clean finish does not mean your artifact read everything.

Before You Open the Pull Request

CI runs these checks on pull requests. The lint step at the end needs `pylint`, which `requirements.txt` does not list, so install it first with `pip install pylint`. Then run the checks from the repository root before you push:

```
python admin/scripts/check_claim_language.py
python admin/scripts/check_artifact_descriptions.py
python admin/scripts/check_artifact_paths.py
python admin/scripts/check_conversation_column_order.py
python admin/scripts/check_container_markers.py
python admin/scripts/check_html_safety.py
python admin/scripts/check_report_local_paths.py
python admin/scripts/check_source_path.py
python admin/scripts/validate_sample_data.py
python admin/scripts/lint_changed.py --base-ref "$(git merge-base origin/main HEAD)" \
    scripts/artifacts/myModule.py
```

`check_claim_language.py` flags wording in your artifact's examiner-facing text that claims more than the data can show. The lint step fails on any new `pylint` warning in the files you changed. CI also replays the recorded test cases with `admin/test/scripts/run_test_cases.py`.

If your pull request changes a module and includes no test data, a bot comments asking for some. `admin/docs/testing/create_module_test_cases.md` explains how test cases are built.

Submitting a Pull Request

1. **Fork** the iLEAPP repository on GitHub.
2. **Create a branch** named for your artifact: `git checkout -b artifact/my-app-messages`
3. **Place your module** in `scripts/artifacts/myModule.py`
4. **Test** against a real extraction. Confirm data appears correctly in both the HTML report and LAVA.
5. **Run the checks** from the section above.
6. **Commit** with a clear message: `Add MyApp message parser`
7. **Open a pull request** against the main branch of `abrignoni/iLEAPP`

PR checklist:

- [] `__artifacts_v2__` sits at the top of the file, and its key matches the function name exactly
- [] description and notes say only what the data shows
- [] All timestamps are converted to UTC `datetime` values
- [] `data_headers` types are set for `datetime`, `date`, `media`, and `phonenumbers` columns, and no other type names are used
- [] Media columns hold `check_in_media` references, not file paths
- [] The third return value is the real path of every file read, one per line
- [] Any path placed in a row goes through `context.get_relative_path()`
- [] `output_types` is set to "standard" unless there is a specific reason otherwise
- [] `artifact_icon` is a Tabler icon name
- [] Conversation `data_views` added if the artifact contains messages, with the columns in the required order
- [] Module tested against a real extraction, and the checks above run clean
- [] Author field populated in `__artifacts_v2__`

Key Helper Functions Reference

Helpers are in `scripts/ilapfuncs.py`. The two context methods are available on the context your function receives.

Function	Use
<code>context.get_files_found()</code>	Every file your paths patterns matched
<code>context.get_relative_path(path)</code>	A matched file's path inside the extraction
<code>get_file_path(files_found, filename)</code>	The first matched file whose name matches <code>filename</code> , or <code>None</code> . <code>filename</code> can include parent folders, as in <code>Attachments/photo.jpg</code>
<code>get_sqlite_db_records(path, query)</code>	Run a query on a database opened read-only and return the rows
<code>does_table_exist_in_db(path, table)</code>	Check for a table before querying it
<code>does_column_exist_in_db(path, table, column)</code>	Check for a column before selecting it
<code>get_plist_file_content(file_path)</code>	Read and parse a plist file

Function	Use
<code>check_in_media(file_path, name)</code>	Register a matched file as media and return its reference
<code>check_in_embedded_media(source_file, data, name)</code>	Register media stored as bytes and return its reference
<code>convert_unix_ts_to_utc(ts)</code>	Unix timestamp to a UTC datetime
<code>convert_cocoa_core_data_ts_to_utc(ts)</code>	Cocoa timestamp (seconds since 2001-01-01) to a UTC datetime
<code>convert_plist_date_to_utc(ts)</code>	Plist date to a UTC datetime
<code>convert_ts_human_to_utc(ts)</code>	YYYY-MM-DD HH:MM:SS string to a UTC datetime
<code>logfunc(message)</code>	Write a message to the iLEAPP log